

5-AMALIY ISH

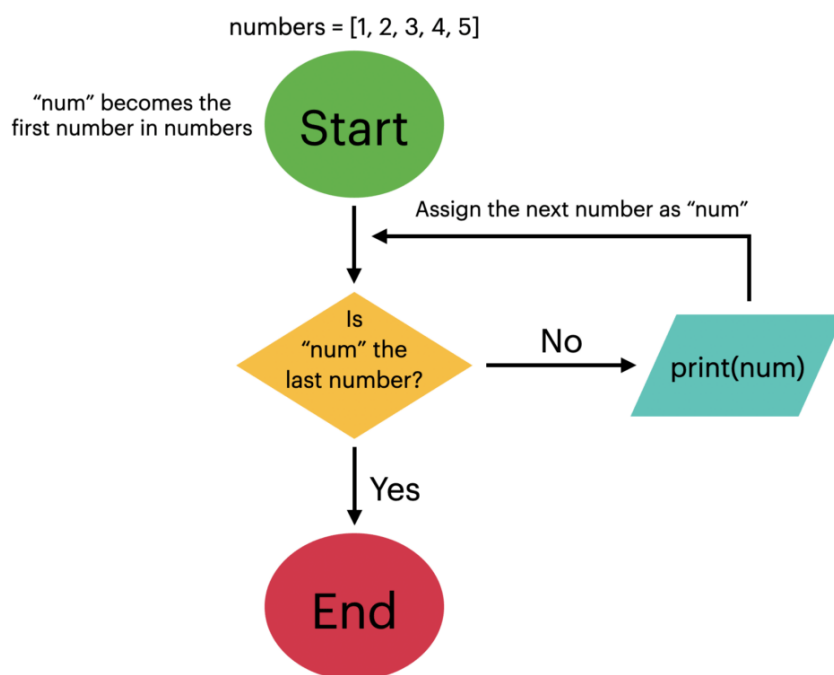
Mavzu: Takrorlanuvchi operatorlar, break, continue operatorlari va ular yordamida dasturlar tuzish.

Ishning maqsadi:

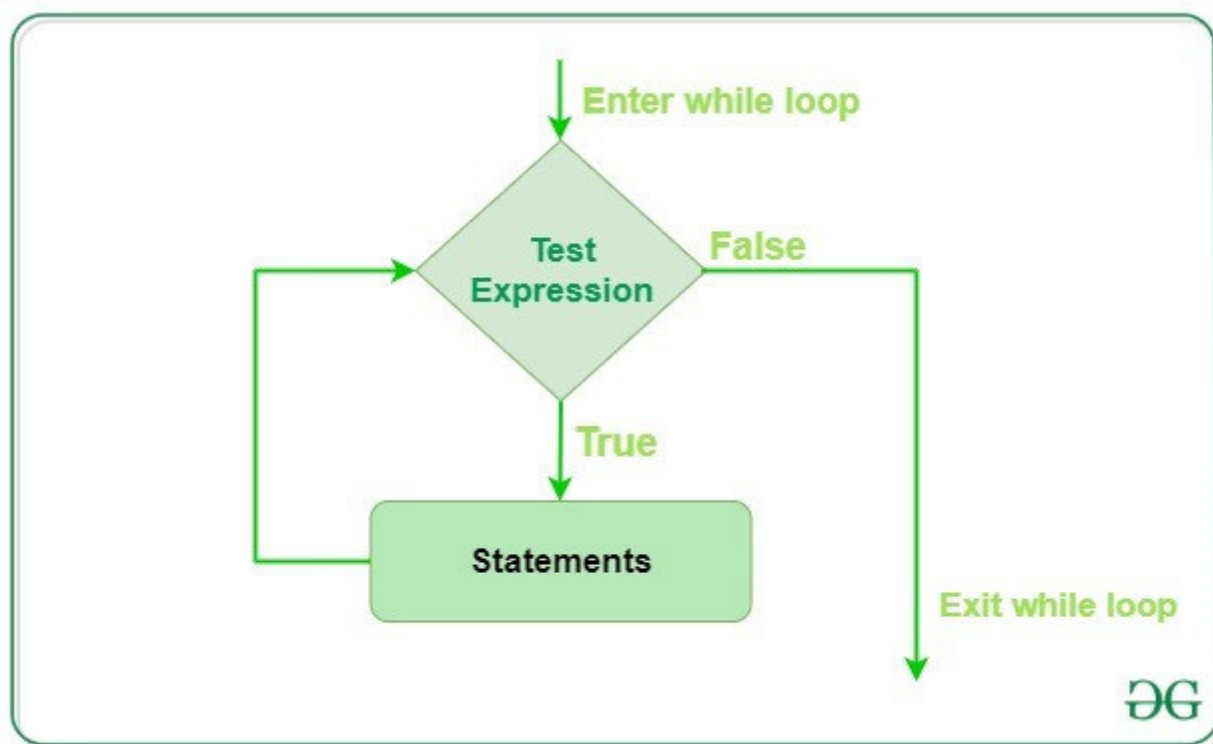
Ushbu amaliy ishning maqsadi talabalarda Python dasturlash tilida takrorlanuvchi jarayonlarni tashkil etish, ya'ni `for` va `while` operatorlari yordamida qayta-qayta bajariladigan hisoblashlarni tuzish bo'yicha amaliy ko'nikmalarni shakllantirishdan iborat. Shuningdek, takrorlanish jarayonini boshqarishda muhim rol o'ynaydigan `break` va `continue` operatorlarining vazifasini tushuntirish, ularni amaliy masalalarda to'g'ri qo'llash malakasini hosil qilish ko'zda tutiladi. Amaliy ish yakunida talabalar takrorlanish shartlarini to'g'ri qo'yish, cheksiz sikl holatlarini oldini olish hamda algoritmni samarali ishlashiga erishish kutiladi.

Nazariy qism:

Takrorlanuvchi operatorlar dasturlashda eng ko'p qo'llaniladigan mantiqiy tuzilmalardan biridir. Ko'plab muhandislik masalalarida hisoblashlar bir marta emas, balki bir necha marta takror bajarilishi kerak bo'ladi. Masalan, ketma-ketlik yig'indisini topish, jadval ko'rinishida qiymatlar hisoblash, sensor ma'lumotlarini doimiy nazorat qilish, aniqlik talabiga yetguncha iteratsiya bajarish kabi jarayonlar takrorlanishsiz amalga oshmaydi. Shu sababli takrorlanuvchi operatorlarni puxta o'zlashtirish dasturlash asoslarining eng muhim qismidir.

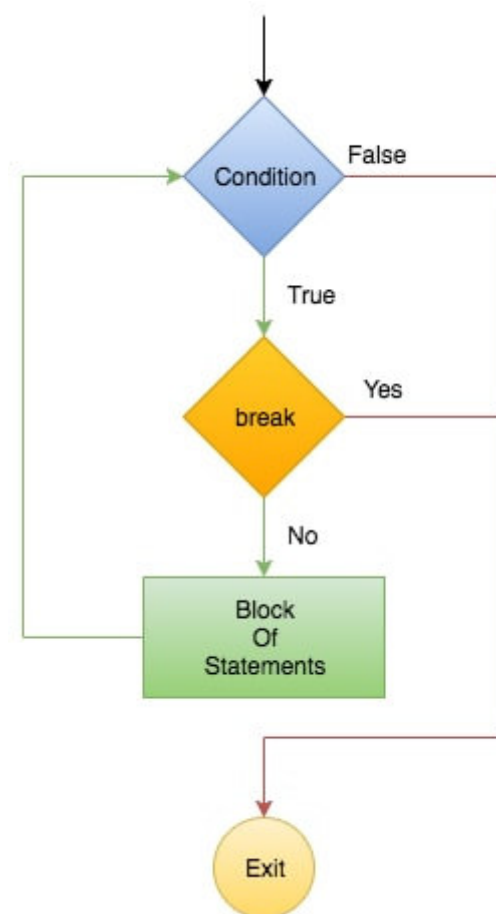


Python tilida takrorlanish asosan ikki xil operator orqali tashkil etiladi: `for` va `while`. `for` operatori ko‘pincha takrorlanishlar soni oldindan ma’lum bo‘lgan holatlarda ishlatiladi. Masalan, 1 dan 10 gacha sonlarni chiqarish, massiv elementlarini ketma-ket ko‘rib chiqish, aniq miqdordagi tajriba natijalarini qayta ishlash kabi vaziyatlarda `for` juda qulay. Python’da `for` odatda `range()` funksiyasi bilan birga qo‘llanadi. `range(a, b)` ifodasi `a` dan `b-1` gacha bo‘lgan qiymatlarni ketma-ket beradi. Bu dastur tuzishda takrorlanish chegarasini aniq belgilash imkonini yaratadi.



`while` operatori esa takrorlanishlar soni oldindan noma’lum bo‘lgan, takrorlanish ma’lum shart bajarilgunga qadar davom etadigan holatlarda qo‘llanadi. Masalan, foydalanuvchi to‘g‘ri parol kiritmaguncha so‘rash, xatolik belgilangan qiymatdan kichik bo‘lguncha iteratsiya qilish, sensor qiymati normal holatga kelguncha tekshirish kabi vazifalarda `while` sikli juda muhim hisoblanadi. `while` operatorida takrorlanish sharti har safar tekshiriladi. Shart `True` bo‘lsa, sikl davom etadi; shart `False` bo‘lsa, sikl tugaydi.

Takrorlanish operatorlari bilan ishlashda eng ko‘p uchraydigan muammolardan biri — cheksiz (infinite) sikl. Bu holat, odatda, sikl sharti hech qachon `False` bo‘lmaydigan tarzda yozilganda yoki boshqaruvchi o‘zgaruvchi yangilanmay qolganda yuz beradi. Cheksiz sikl dastur ishlashini to‘xtatib qo‘yishi, resurslarni ortiqcha sarflashi va tizimni sekinlashtirishi mumkin. Shu sababli sikl sharti va yangilanish qismi doim e’tiborda bo‘lishi kerak.



Takrorlanishni boshqarishda `break` va `continue` operatorlari alohida ahamiyatga ega. `break` operatori siklni majburiy ravishda to'xtatish uchun ishlatiladi. Ya'ni sikl ichida ma'lum bir holat yuzaga kelganda, dastur takrorlanishni davom ettirmasdan darhol sikldan chiqadi. Masalan, qidirilayotgan qiymat topilganda siklni darhol to'xtatish, foydalanuvchi "stop" deb yozganda jarayonni yakunlash, yoki xavfli holat aniqlanganda monitoringni to'xtatish kabi vaziyatlar `break` orqali amalga oshiriladi.

`continue` operatori esa siklni to'xtatmaydi, lekin joriy takrorlanishdagi qolgan buyruqlarni tashlab, siklni keyingi takrorlanishdan davom ettiradi. Bu operator ma'lum shart bajarilganda ayrim qiymatlarni "o'tkazib yuborish" uchun juda qulay. Masalan, manfiy sonlarni hisobga olmaslik, faqat juft sonlar bilan ishlash, yoki xato kiritilgan qiymatlarni qayta so'rash jarayonlarida `continue` foydali bo'ladi.

Takrorlanish operatorlari, `break`, `continue` yordamida tuzilgan dasturlar ancha ixcham, mantiqiy va samarali ishlaydi. Ayniqsa, katta hajmli ma'lumotlarni qayta ishlashda bu operatorlar dastur unumdorligini oshiradi.

Amaliy ishni bajarilishi:

Amaliy ish davomida talaba Python muhitida yangi dastur faylini yaratadi va takrorlanuvchi operatorlar bilan ishlashga oid bir nechta misollarni bosqichma-bosqich bajaradi. Dastlab `for` sikli yordamida oddiy takrorlanishlar, masalan, berilgan oraliqdagi sonlarni chiqarish va yig'indi hisoblash kabi masalalar yechiladi. Keyin `while` sikli yordamida shart bajarilgunga qadar takrorlanadigan jarayonlar tashkil etiladi.

Shundan so'ng sikl ichida `break` operatori qo'llanadigan vaziyatlar ko'rib chiqiladi. Masalan, ma'lum qiymat topilganda siklni to'xtatish yoki foydalanuvchi buyruq berganda jarayonni yakunlash kabi holatlar tahlil qilinadi. Keyingi qismda `continue` operatori yordamida ayrim takrorlanishlarni o'tkazib yuborish, ya'ni shartga mos bo'lmagan qiymatlarni hisobga olmaslik kabi amaliy misollar bajariladi.

Ish yakunida talaba tuzgan dasturlarini ishga tushirib tekshiradi, natijalarni tahlil qiladi, xatolar yuzaga kelgan bo'lsa ularni tuzatadi va yakuniy xulosa yozadi.

Talabalar uchun individual topshiriqlar:

Birinchi topshiriq:

1. 1 dan n gacha juft sonlar yig'indisini hisoblang (`continue` bilan toq sonlarni o'tkazing).
2. Kiritilgan n sonigacha bo'lgan sonlar ichidan 3 ga karralilarni chiqarish.
3. Foydalanuvchi 0 kiritmaguncha sonlarni qabul qilib, ularning yig'indisini topish (`while`).
4. Berilgan son tub yoki tub emasligini tekshirish (topilganda `break`).
5. n ta son ichidan eng kattasini topish (`while` yoki `for`).
6. Foydalanuvchi "stop" yozmaguncha kiritilgan matnlarni chiqarish (`break`).
7. 1 dan n gacha sonlar ko'paytmasini (faktorial) hisoblash.
8. 1 dan n gacha faqat 5 ga karrali sonlarni chiqarish (`continue`).
9. Berilgan oraliqda (a dan b gacha) manfiy sonlarni hisobga olmasdan yig'indi topish.
10. Kiritilgan sonlar ichidan faqat musbatlarini sanash (`continue` bilan manfiylarni o'tkazish).
11. Parol tekshirish: to'g'ri parol kiritilmaguncha qayta so'rash (`while`).
12. 1 dan n gacha sonlar ichida birinchi bo'lib 7 ga bo'linadigan sonni topish (`break`).
13. Fibonacci ketma-ketligidan n ta hadni chiqarish.
14. Kiritilgan sonlar orasidan 0 chiqsa, siklni to'xtatish (`break`).
15. 1 dan n gacha bo'lgan sonlardan faqat toqlarini chiqarish (`continue` bilan juftni o'tkazish).

16. a dan b gacha sonlar orasida nechta juft borligini hisoblash.
17. Kiritilgan n uchun bo'luvchilarini topish (0 ga bo'lishdan ehtiyot bo'ling).
18. Berilgan massiv (ro'yxat) ichidan manfiy elementlarni chiqarib tashlab, qolganlarini chiqarish (continue).
19. Kiritilgan sonning raqamlari yig'indisini topish (while).
20. Kiritilgan sonning raqamlari sonini aniqlash (while).
21. Berilgan matnda birinchi "a" harfi uchraganda to'xtash (break).
22. 1 dan n gacha sonlar orasida faqat tub sonlarni chiqarish.
23. Kiritilgan n gacha bo'lgan sonlar ichida kvadrat sonlarni chiqarish.
24. Kiritilgan sonlar ichida eng kichigini topish.
25. Foydalanuvchi manfiy son kiritisa, uni hisobga olmasdan davom etish (continue).
26. Kiritilgan sonlar orasidan faqat 10 dan katta bo'lganlarini sanash.
27. Kiritilgan n uchun $1+2+\dots+n$ yig'indini while bilan hisoblash.
28. n sonini kiritib, 1 dan n gacha sonlarni teskari chiqarish.
29. Berilgan oraliqda 4 ga bo'linmaydigan sonlarni chiqarish (continue).
30. Kiritilgan son 100 dan oshsa, siklni darhol to'xtatish (break).

Qo'shimcha topshiriqlar:

- 1) 1 dan 10 gacha bo'lgan sonlarni for sikli yordamida chiqaring

Izoh: range(1, 11) orqali 1..10 chiqariladi.

```
for i in range(1, 11):  
    print(i)
```

- 2) 1 dan n gacha bo'lgan sonlar yig'indisini for bilan hisoblang

Izoh: n kiritiladi, sikl yig'indini yig'adi.

```
n = int(input("n = "))  
s = 0  
for i in range(1, n + 1):  
    s += i  
print("Yig'indi =", s)
```

- 3) 1 dan 20 gacha bo'lgan juft sonlarni chiqaring

Izoh: 2 qadam bilan yuriladi.

```
for i in range(2, 21, 2):  
    print(i)
```

4) while sikli yordamida 10 dan 1 gacha sanang

Izoh: x kamayib boradi.

```
x = 10
while x >= 1:
    print(x)
    x -= 1
```

5) Foydalanuvchi 0 kiritmaguncha sonlarni qabul qiling

Izoh: 0 kiritilganda break bilan to'xtaydi.

```
while True:
    x = int(input("Son kiriting (to'xtash: 0): "))
    if x == 0:
        break
    print("Kiritildi:", x)
```

6) Massiv elementlarini for sikli yordamida chiqaring

Izoh: ro'yxat elementlari ketma-ket chiqariladi.

```
a = [3, 7, -2, 9, 0]
for x in a:
    print(x)
```

7) Massivdagi musbat sonlar sonini aniqlang

Izoh: shart bilan sanaladi.

```
a = [3, 7, -2, 9, 0, -5, 4]
cnt = 0
for x in a:
    if x > 0:
        cnt += 1
print("Musbatlar soni =", cnt)
```

8) break bilan 1 dan 100 gacha, 50 da to'xtasin

Izoh: 50 ga kelganda sikl tugaydi.

```
for i in range(1, 101):
```

```
if i == 50:
    break
print(i)
```

9) Massivda berilgan son topilganda siklni to'xtating

Izoh: topilsa "topildi" chiqib, break ishlaydi.

```
a = [5, 2, 9, 1, 7]
target = int(input("Qidiriladigan son: "))

for x in a:
    if x == target:
        print("Topildi:", x)
        break
else:
    print("Topilmadi")
```

10) continue yordamida manfiy sonlarni o'tkazib yuboring

Izoh: manfiy bo'lsa, keyingi qadamga o'tadi.

```
a = [3, -1, 5, -7, 2]
for x in a:
    if x < 0:
        continue
    print("Musbat:", x)
```

11) 1 dan 30 gacha bo'lgan toq sonlarni chiqaring

Izoh: 1 dan boshlab 2 qadam.

```
for i in range(1, 31, 2):
    print(i)
```

12) continue bilan 5 ga karrali sonlarni chiqarmang

Izoh: 5 ga bo'linsa o'tkazib yuboriladi.

```
for i in range(1, 51):
    if i % 5 == 0:
        continue
    print(i)
```

13) To'g'ri parol kiritilmaguncha so'rash (while)

Izoh: parol mos bo'lsa sikl tugaydi.

```
correct = "1234"
while True:
    p = input("Parolni kiriting: ")
    if p == correct:
        print("Kirish muvaffaqiyatli!")
        break
    print("Noto'g'ri, qayta urinib ko'ring.")
```

14) Massivdagi eng katta elementni sikl bilan toping

Izoh: max ishlatmay, qo'lda topiladi.

```
a = [3, 7, -2, 9, 0]
m = a[0]
for x in a:
    if x > m:
        m = x
print("Eng katta =", m)
```

15) Massiv elementlari yig'indisini hisoblang

Izoh: s yig'ilib boradi.

```
a = [3, 7, -2, 9, 0]
s = 0
for x in a:
    s += x
print("Yig'indi =", s)
```

16) break bilan birinchi manfiy son topilganda to'xtating

Izoh: manfiy chiqsa sikl tugaydi.

```
a = [4, 8, 2, -3, 7, -1]
for x in a:
    if x < 0:
        print("Birinchi manfiy topildi:", x)
        break
    print("Musbat:", x)
```


17) 1 dan n gacha bo'lgan sonlar kvadratini chiqaring

Izoh: $i*i$ hisoblanadi.

```
n = int(input("n = "))
for i in range(1, n + 1):
    print(i, "kvadrati =", i*i)
```

18) for siklida qadam (step) bilan ishlash misoli

Izoh: 0 dan 20 gacha 4 qadam bilan.

```
for i in range(0, 21, 4):
    print(i)
```

19) while bilan raqamlar sonini aniqlang

Izoh: son 10 ga bo'linib kamayadi, sanagich oshadi.

```
n = int(input("n = "))
n = abs(n)
if n == 0:
    print("Raqamlar soni = 1")
else:
    cnt = 0
    while n > 0:
        n //= 10
        cnt += 1
    print("Raqamlar soni =", cnt)
```

20) Kiritilgan son tub yoki tub emasligini aniqlang

Izoh: bo'luvchilar tekshiriladi.

```
n = int(input("n = "))

if n < 2:
    print("Tub emas")
else:
    is_prime = True
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            is_prime = False
            break
```

```
print("Tub" if is_prime else "Tub emas")
```

21) continue bilan 0 ga teng sonlarni hisobga olmang

Izoh: 0 bo'lsa o'tkazib yuboriladi, qolganlar yig'iladi.

```
a = [3, 0, 5, 0, -2, 7]
s = 0
for x in a:
    if x == 0:
        continue
    s += x
print("0 larsiz yig'indi =", s)
```

22) 1..100 ichidan faqat 3 ga karralilarni chiqaring

Izoh: if i%3==0 sharti.

```
for i in range(1, 101):
    if i % 3 == 0:
        print(i)
```

23) break va continue birga ishlatilgan dastur

Izoh: manfiylarni o'tkazadi (continue), 0 bo'lsa to'xtaydi (break).

```
while True:
    x = int(input("Son kiriting (0 to'xtatadi): "))
    if x == 0:
        break
    if x < 0:
        continue
    print("Qabul qilindi:", x)
```

24) Massivdagi faqat juft sonlarni chiqaring

Izoh: x%2==0 bo'lsa chiqaradi.

```
a = [3, 6, 7, 8, 11, 12]
for x in a:
    if x % 2 == 0:
        print(x)
```

25) Massiv elementlarini teskari tartibda chiqaring

Izoh: reversed yoki indeks bilan chiqarish mumkin.

```
a = [3, 7, -2, 9, 0]
for x in reversed(a):
    print(x)
```

26) while yordamida sonlar yig'indisini hisoblang

Izoh: 1..n yig'indisini while bilan.

```
n = int(input("n = "))
i = 1
s = 0
while i <= n:
    s += i
    i += 1
print("Yig'indi =", s)
```

27) for sikli yordamida stringni harflab chiqaring

Izoh: har bir belgi alohida chiqariladi.

```
text = input("Matn kiriting: ")
for ch in text:
    print(ch)
```

28) Sikl ichida foydalanuvchidan kiritish qabul qiluvchi dastur

Izoh: n marta qiymat kiritib ro'yxatga to'playdi.

```
n = int(input("Nechta son kiritasiz? "))
a = []
for i in range(n):
    a.append(int(input(f"{i+1}-son: ")))
print("Ro'yxat:", a)
```

29) Cheksiz siklga misol va to'xtatish usuli

Izoh: while True cheksiz ishlaydi, break bilan to'xtatamiz.

```
while True:
    cmd = input("Chiqish uchun 'q' yozing: ")
```

```
if cmd == "q":  
    break  
print("Davom etyapti...")
```

30) Takrorlanuvchi operatorlarning ahamiyati haqida qisqa izoh yozing

Izoh: bu kod emas, talaba 4–6 jumla yozadi. Namuna matn:

Nazorat savollari:

1. Python'da takrorlanuvchi operatorlar deganda nimani tushunasiz?
2. `for` sikli qaysi vaziyatlarda qo'llaniladi?
3. `while` sikli qaysi vaziyatlarda qulayroq hisoblanadi?
4. `range()` funksiyasi qanday ishlaydi va qayerda qo'llaniladi?
5. Cheksiz sikl (infinite loop) nima va u qanday yuzaga keladi?
6. `break` operatorining vazifasi nimadan iborat?
7. `continue` operatorining vazifasi nimadan iborat?
8. `break` va `continue` operatorlarining farqi nimada?
9. Takrorlanish sharti noto'g'ri berilsa qanday muammolar yuzaga keladi?
10. Muhandislik masalalarida takrorlanish algoritmlariga qaysi misollarni keltirish mumkin?